FreeRTOS Projects Q&A

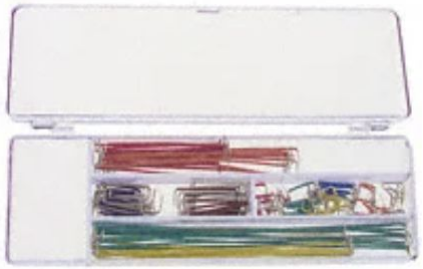
Mark Van Genderen

Rough FreeRTOS Project Outline

- Your HW
- FreeRTOS
- Scheduling Algorithm
- Implementation
- Testing
- Debugging

Your HW

What you get:



- Raspberry Pi Pico H
- Breadboard (Cheap)
  - If you own a good breadboard, maybe use that instead
- Raspberry Pi Debug Probe
- Jumper Wires
- 1x Yellow, 1x Red, & 1x Green LED
- 1x Button Switch
- 1x 10k, 4x 220 Ohm Resistor
- Micro USB Cable
- ESD Bag to store electronics

Setting up HW

Setup

- See setup guide “FreeRTOS_on_RPi_Pico.pdf”
  - Will be uploaded to canvas
  - Outlines various resources for getting environment setup, and running your first program on Pico

FreeRTOS

What is it?

FreeRTOS Documentation

- https://www.freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-RTOS_book
  - Honestly worth a read
  - It is long (about 300 pages), but very readable (a lot if its photos and code examples)
  - I recommend that everyone reads CH2-8
    - 2 The FreeRTOS Kernel Distribution
    - 3 Heap Memory Management
    - 4 Task Management
    - 5 Queue Management
    - 6 Software Timer Management
    - 7 Interrupt Management
    - 8 Resource Management

Scheduling Algorithm

Understand your algorithm

- Read the paper
- Some of the algorithms have very clear explanation in the course textbook
- We should have class discussions/presentations for some of the algorithms

Relevant Resources

- Earliest Deadline First (EDF)
  - CH4.4 of textbook
  - EDF Lecture Slides
  - Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment
    - Paper which introduced EDF (but they did not name the algorithm EDF in this paper)
    - Probably interesting, but by no means is it a necessary read
- Stack Resource Policy (SRP)
  - CH7.8 of textbook
  - SRP Lecture Slides
  - Stack-based scheduling of realtime processes
- Constant Bandwidth Server (CBS)
  - CH6.9 of textbook
  - CBS Lecture Slides
  - Integrating multimedia applications in hard real-time systems
    - Very Useful. Provides a lot of Pseudocode
- Multiprocessor Support (15%)
  - FreeRTOS AMP & SMP
  - Module 2 Lecture Slides
  - Multiprocessor Scheduling for Real-Time Systems textbook

Implementation

Overview of implementation steps

- Clone FreeRTOS repo
- Familiarize yourself with the repo
- Determine your implementation methodology (i.e. how you will change the existing repo to enable your scheduling algorithm)
- Code!

Some General Implementation Recommendations

- Static is Best!!
  - Avoid dynamic memory where possible
  - Prevents bugs
  - Easier debugging
- Never busy wait
  - It’s better to allow other tasks to run while your current task is waiting
- Use non-blocking functions when possible
  - Leads to fewer context switches and a more consistent task computation time

Testing

Overview of Testing Steps

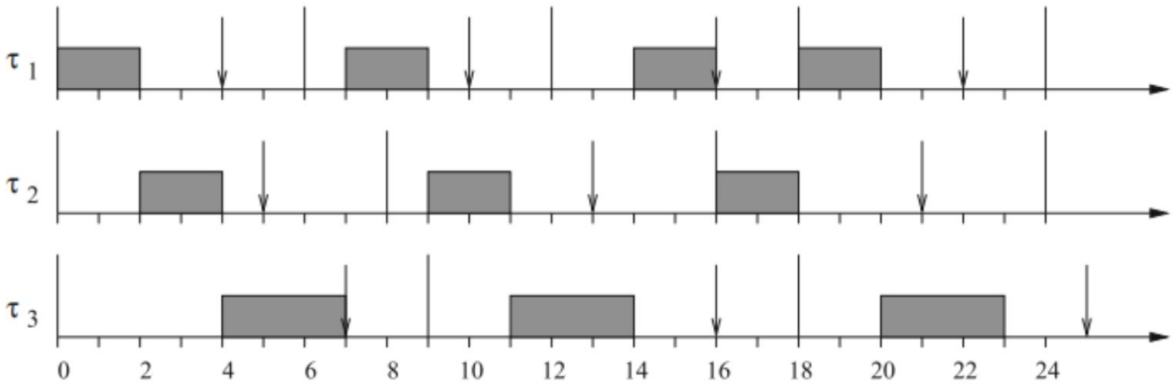
- Setup basic test case
- Make test results observable (i.e. using led, logic analyzer, ...)
- Once your initial tests passes, keep adding test cases until your confident you have proven the efficacy of your implementation

Logic Analyzers

- Saleae
  - Professional standard
  - Expensive
  - ~700 CAD
  - https://saleae.com/logic
- Digilent
  - Good value
  - A lot of IOs
  - ~400 CAD
  - https://www.digikey.ca/en/products/detail/digilent-inc/240-127/12696371
- 3rd party
  - Cheap knock-off which can use the Saleae Logic analyzer software
  - Infrequently receives errors, but works well in general
  - ~16 CAD
  - Amazon Link

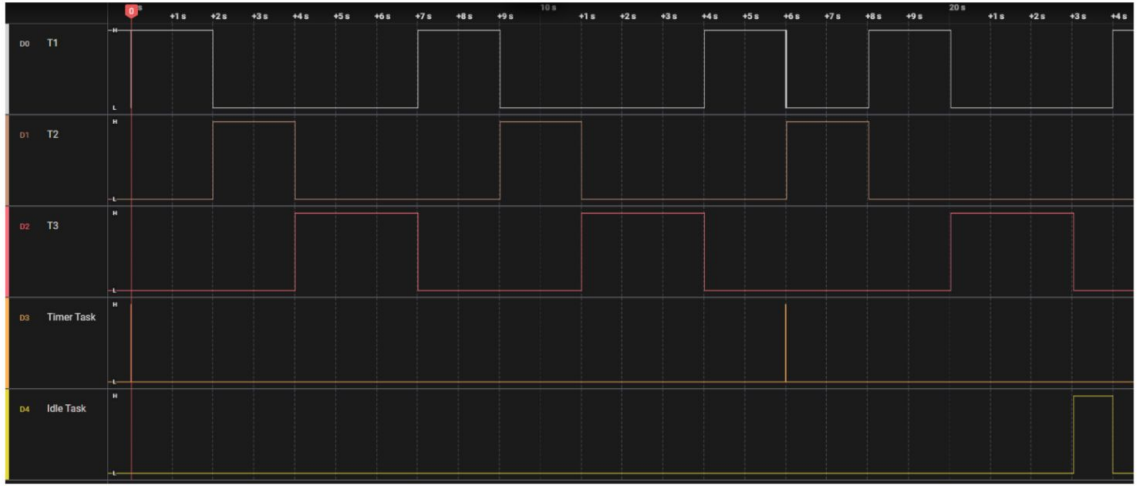
EDF Test Case

	C_i	D_i	T_i
τ_1	2	4	6
τ_2	2	5	8
τ_3	3	7	9



Differences:

Slight overall delay due to inaccurate tick count (caused by the use of critical sections which disable interrupts and therefore delay ticks)



Debugging

Most of your time will likely be spent debugging!:(!

Trace Hook Macros!

- Macros that get called when a task being selected to start or stop running
  - We can use these macros to now which task is running at any given time
- In real time systems, printouts are very slow!
- Toggling GPIOS is the best way to communicate quickly
  - We then observe these GPIOS via LEDs, or Logic Analyzer, or …
- FreeRTOS Trace Hook Macros

Recommendations

Overall Recommendations

- Start Early
- Expect ~half your time to be spent debugging/testing
- Have a reliable debugging methodology (i.e. logic analyzer)
- Wire reliably
  - A loose wire can lead to hours of debugging. Try to make your setup fairly robust (i.e. keep it in a box, keep your wires flush with the breadboard when possible, etc.)
- Don’t forget to monitor tasks that you haven’t explicitly created, but which exist in FreeRTOS
  - I.e idle task, and timer task (if your using a timer)

Q&A

What FreeRTOS Project related topics do you want covered in future presentations?